

Gemini Prompt — Identity Sprawl & Privilege Abuse Detection System (Full Skeleton)

CONTEXT

You are helping build a hackathon project for the "Identity Sprawl & Privilege Abuse Detection" challenge. The system detects risky privilege patterns across a hybrid identity environment, classifies accounts by risk level, and generates LLM-powered explanations with a Devil's Advocate layer to reduce false positives.

Two developers are working in parallel. You are building the complete skeleton of the entire system — every file, every folder, every function — wired end to end and working with stubs. Real logic will be filled in later by the two developers independently. The skeleton must be complete enough that the app runs, the frontend renders data, and the backend responds to every API call — all using stub data that has the exact same shape as real data will have.

Do not simplify the structure. Do not merge files. Do not skip any file mentioned. Every file must exist, every function must exist, every stub must return correctly shaped data.

TECH STACK

Backend

- **Python 3.11+**
- **FastAPI with Uvicorn** — REST API server
- **Pandas** — CSV loading and data manipulation
- **scikit-learn** — ML models (Isolation Forest, KMeans) — stubbed for now
- **Anthropic Python SDK** — LLM calls — stubbed for now
- **No database** — persist everything to JSON flat files on disk

Frontend

- **React 18** with **Vite**
- **Tailwind CSS** — all styling
- **Recharts** — charts and visualizations
- **lucide-react** — icons
- **No Redux, no React Router** — single page app, useState only

Communication

- Frontend polls `GET /status` every 5 seconds to check pipeline state

- All other calls are standard REST fetch from `api.js`
 - LLM narratives stream token by token using FastAPI `StreamingResponse` + frontend `EventSource`
-

FOLDER STRUCTURE

Create exactly this structure. Do not deviate:

```

project/
├─ backend/
│   ├─ main.py
│   ├─ pipeline.py
│   └─ models/
│       ├─ __init__.py
│       ├─ isolation_forest.py
│       ├─ kmeans_events.py
│       ├─ org_anomaly.py
│       ├─ sod_violations.py
│       └─ compliance.py
│   └─ llm/
│       ├─ __init__.py
│       └─ narrator.py
│   └─ schemas.py
│   └─ data_loader.py
│   └─ data/
│       ├─ identity_users.csv      ← already exists, do not create
│       ├─ identity_events.csv    ← already exists, do not create
│       ├─ user_profiles.json     ← generated at runtime
│       └─ feedback.json         ← generated at runtime
└─ frontend/
    ├─ index.html
    ├─ vite.config.js
    ├─ tailwind.config.js
    ├─ postcss.config.js
    ├─ package.json
    └─ src/
        ├─ main.jsx
        ├─ App.jsx
        ├─ api.js
        └─ components/
            ├─ RiskTable.jsx
            ├─ UserCard.jsx
            ├─ NarrativePanel.jsx
            ├─ OrgAnomalyPanel.jsx
            ├─ FeedbackButtons.jsx
            ├─ BreachImpactPanel.jsx
            ├─ CompliancePanel.jsx
            └─ StatusBar.jsx

```

CENTRAL DATA MODEL

This is the most important part. Every file in the system reads from or writes to this shape.

Do not change this shape anywhere. Add comments in every file explaining each field.

python

```

# schemas.py – define this as a Python TypedDict AND as a plain dict template
# Also export a JSON-serializable version

UserRiskProfile = {
    # --- Identity fields (from identity_users.csv) ---
    "user_id": str,          # e.g. "USR00045"
    "username": str,         # e.g. "pooja.murphy"
    "email": str,
    "department": str,       # e.g. "Finance", "Engineering"
    "job_title": str,
    "privilege_level": str,   # "user" | "power-user" | "admin" | "service-
    "systems_access": list[str], # e.g. ["AD", "AWS_IAM", "Salesforce"] – spli
    "last_login": str,       # ISO date string
    "days_inactive": int,
    "is_active": bool,
    "hire_date": str,        # ISO date string
    "is_new_hire": bool,     # True if hired within last 30 days – compute

    # --- ML output fields (from pipeline.py) ---
    "risk_score": float,     # 0.0 to 100.0 – higher = riskier
    "anomaly_score_raw": float, # raw Isolation Forest score before normaliza
    "cluster_label": int,     # KMeans cluster id from event clustering
    "cluster_description": str, # human label for the cluster e.g. "High-freq

    # --- Rule-based flags (from models/) ---
    "flags": list[str],      # list of flag codes e.g. ["STALE_ADMIN", "SO
    "sod_violations": list[dict], # list of {role_a, role_b, explanation} dicts
    "compliance_gaps": list[dict], # list of {framework, control, gap, severity}
    "org_anomaly": dict,     # {is_anomaly: bool, reason: str, dept_baseli

    # --- Event summary (aggregated from identity_events.csv) ---
    "event_count": int,
    "flagged_events": list[dict], # subset of events that triggered flags – ful
    "after_hours_ratio": float, # fraction of events outside business hours
    "high_sensitivity_ratio": float,
    "fail_ratio": float,
    "export_count": int,
    "admin_op_count": int,

    # --- LLM output fields (from llm/narrator.py) ---
    # These are None until the user's card is clicked in the UI
    "narrative": {
        "prosecution": str,    # LLM-generated accusation with evidence
        "devils_advocate": str, # LLM-generated challenge to the accusation
        "doubt_score": float,   # 0.0 (certain threat) to 1.0 (almost certain
        "final_severity": str,  # "CRITICAL" | "HIGH" | "REVIEW" | "INFORMATI

```

```

        "recommendation": str,      # Actionable step e.g. "Disable account pending"
        "breach_impact": str,      # If compromised, what systems/data are at risk
    },                             # OR None if not yet generated

    # --- Feedback loop fields ---
    "suppressed": bool,            # True if analyst marked as false positive
    "feedback_note": str,          # Optional analyst note when suppressing
}

```

FILE-BY-FILE INSTRUCTIONS

backend/schemas.py

Define the following:

1. `UserRiskProfile` as a Python `TypedDict` with every field above and inline comments on each field
2. A function `empty_narrative() -> dict` that returns a narrative dict with all fields as empty strings and `doubt_score` as 0.5
3. A function `make_stub_profile(user_row: dict) -> dict` that takes a raw CSV row dict and returns a fully populated `UserRiskProfile` with:
 - Real values for all identity fields (parse `systems_access` by splitting on `|`)
 - `is_new_hire` computed as True if `hire_date` is within 30 days of today
 - All ML fields set to stub values: `risk_score=50.0`, `anomaly_score_raw=0.0`, `cluster_label=0`, `cluster_description="Unclustered"`
 - Empty flags list, empty `sod_violations`, empty `compliance_gaps`
 - `org_anomaly` set to `{is_anomaly: False, reason: "", dept_baseline: {}}`
 - `event_count=0`, `flagged_events=[]`, all ratios 0.0
 - `narrative` set to None
 - `suppressed=False`, `feedback_note=""`
4. Add a comment at the top: `# THIS FILE IS THE CONTRACT. DO NOT CHANGE FIELD NAMES OR TYPES WITHOUT UPDATING ALL OTHER FILES.`

backend/data_loader.py

Define the following functions with comments:

```
python
```

```
def load_users() -> list[dict]:
    # Reads identity_users.csv using pandas
    # Converts each row to a plain dict
    # Parses systems_access by splitting on '|' into a list
    # Parses is_active as bool
    # Returns list of raw user dicts (NOT UserRiskProfile yet)
    # STUB: reads real CSV, returns real rows unmodified
    pass

def load_events() -> list[dict]:
    # Reads identity_events.csv using pandas
    # Converts each row to a plain dict
    # Returns list of raw event dicts
    # STUB: reads real CSV, returns real rows unmodified
    pass

def get_events_for_user(user_id: str, events: list[dict]) -> list[dict]:
    # Filters events list to only events matching user_id
    # Returns list of event dicts
    pass

def save_profiles(profiles: list[dict]) -> None:
    # Writes list of UserRiskProfile dicts to data/user_profiles.json
    # Pretty printed JSON
    pass

def load_profiles() -> list[dict]:
    # Reads data/user_profiles.json
    # Returns list of UserRiskProfile dicts
    # Returns empty list if file does not exist
    pass

def save_feedback(feedback: dict) -> None:
    # Writes feedback dict to data/feedback.json
    # feedback is keyed by user_id: { "USR00045": { suppressed: true, note: "..
    pass

def load_feedback() -> dict:
    # Reads data/feedback.json
    # Returns dict keyed by user_id
    # Returns empty dict if file does not exist
    pass
```

backend/models/isolation_forest.py

python

```
def compute_user_features(user_profile: dict, events: list[dict]) -> dict:
    # Takes a UserRiskProfile dict and that user's events list
    # Computes and returns a flat feature dict:
    # {
    #     privilege_score: int,          # user=1, power-user=2, admin=3, service-a
    #     systems_count: int,           # len(systems_access)
    #     days_inactive: int,
    #     event_count: int,
    #     after_hours_ratio: float,     # events with time_classification in (nigh
    #     high_sensitivity_ratio: float,
    #     fail_ratio: float,
    #     admin_op_count: int,
    #     export_count: int,
    # }
    # STUB: return hardcoded feature dict with real values computed from actual
    # This function does NOT need to be stubbed – compute real features, only t
    pass

def score_users(profiles: list[dict], all_events: list[dict]) -> list[dict]:
    # Takes all UserRiskProfile dicts and all events
    # For each profile:
    #     - calls compute_user_features to get feature dict
    #     - runs Isolation Forest (STUBBED: just set risk_score = random float 20
    #     - updates profile in place: risk_score, anomaly_score_raw, event_count,
    #       high_sensitivity_ratio, fail_ratio, export_count, admin_op_count, fla
    # Returns updated profiles list
    # STUB NOTE: replace the random score with real IsolationForest.fit_predict
    # Keep compute_user_features real – only the model call is stubbed
    pass
```

backend/models/kmeans_events.py

python

```
def cluster_events(events: list[dict]) -> dict:
    # Takes all events
    # Encodes each event as a feature vector:
    #   action (one-hot or label encoded), resource_sensitivity (low=0,med=1,hi
    #   time_classification (business_hours=0, unusual_hours=1, night=2, weeken
    # Runs KMeans with k=5 (STUBBED: assign random cluster 0-4 to each event)
    # Returns dict keyed by a cluster label string describing each cluster:
    # {
    #   event_id (use timestamp+user_id as key): cluster_label int
    # }
    # STUB NOTE: replace random assignment with real KMeans.fit_predict()
    pass

def assign_cluster_to_users(profiles: list[dict], event_clusters: dict) -> list
    # For each user profile, look at their flagged_events cluster assignments
    # Assign the most common cluster_label among their events to profile.cluste
    # Set cluster_description based on cluster_label:
    #   0: "Normal business hours actor"
    #   1: "High-frequency night actor"
    #   2: "Bulk exporter"
    #   3: "Admin operation heavy"
    #   4: "Mixed anomalous pattern"
    # STUB: assign cluster_label=0 and cluster_description="Normal business hou
    # Updates profiles in place, returns profiles
    pass
```

backend/models/org_anomaly.py

python

```
def compute_department_baselines(profiles: list[dict]) -> dict:
    # Groups all UserRiskProfiles by department
    # For each department computes:
    # {
    #     avg_risk_score: float,
    #     avg_after_hours_ratio: float,
    #     avg_export_count: float,
    #     member_count: int,
    # }
    # Returns dict keyed by department name
    # STUB: compute real averages – this function needs no stubbing, it's pure
    pass

def detect_org_anomalies(profiles: list[dict]) -> list[dict]:
    # Calls compute_department_baselines to get dept baselines
    # For each profile:
    # - compare user's risk_score to their dept avg_risk_score
    # - if user's risk_score > dept_avg + 2*stddev → flag as org anomaly
    # - set profile.org_anomaly = {
    #     is_anomaly: bool,
    #     reason: str,           # e.g. "Risk score 2.3 std deviations above
    #     dept_baseline: dict    # the dept baseline dict for this user's dep
    # }
    # Returns updated profiles
    # STUB: set all org_anomaly.is_anomaly = False with empty reason
    pass
```

backend/models/sod_violations.py

python

```
# Separation of Duties violation detection
# SoD rule: a user should not have BOTH high-privilege access AND perform bulk
# on high-sensitivity resources. Additional rules defined below.

SOD_RULES = [
    # Each rule is a dict:
    # { rule_id, description, condition_fn }
    # condition_fn takes (user_profile, user_events) and returns (bool, explanation)
]

def check_sod_violations(profile: dict, events: list[dict]) -> list[dict]:
    # Runs all SOD_RULES against the user
    # For each triggered rule, appends to violations list:
    # {
    #     rule_id: str,
    #     description: str,
    #     explanation: str,    # specific to this user e.g. "User performed 3 exports
    #     severity: str,      # "HIGH" | "MEDIUM"
    # }
    # Updates profile.sod_violations in place
    # Returns list of violation dicts
    # STUB: return empty list for all users
    # REAL RULES TO IMPLEMENT (leave as comments for teammate):
    # Rule 1: admin privilege_level + export_data on high sensitivity resource
    # Rule 2: performed admin_operation AND export_data in same session (within
    # Rule 3: service-account performing login events (service accounts should
    # Rule 4: user accessing more than 5 distinct systems in a single day
    pass
```

backend/models/compliance.py

python

```
# Maps detected risk patterns to compliance framework violations
# Frameworks: NIST AC-2 (Account Management), GDPR Article 32

COMPLIANCE_CONTROLS = [
    # Each control is a dict:
    # { framework, control_id, control_name, check_fn }
    # check_fn takes (user_profile) and returns (bool violated, str gap_descrip
]

def compute_compliance_gaps(profile: dict) -> list[dict]:
    # Runs all COMPLIANCE_CONTROLS against the user profile
    # For each violated control, appends:
    # {
    #     framework: str,          # "NIST" | "GDPR"
    #     control_id: str,         # e.g. "AC-2"
    #     control_name: str,       # e.g. "Account Management"
    #     gap: str,                # specific gap for this user
    #     severity: str,           # "HIGH" | "MEDIUM" | "LOW"
    # }
    # Updates profile.compliance_gaps in place
    # Returns list of gap dicts
    # STUB: return empty list for all users
    # REAL CONTROLS TO IMPLEMENT (leave as comments for teammate):
    # NIST AC-2: inactive accounts with admin privileges still active
    # NIST AC-2: accounts with no activity review in 30+ days
    # GDPR Art.32: high-privilege users accessing sensitive data without MFA (p
    # NIST AC-6 (Least Privilege): user has more systems_access than typical fo
    pass
```

backend/llm/narrator.py

python

```

# LLM narrative generation using Anthropic SDK
# Implements Devil's Advocate two-section reasoning pattern
# All prompts are defined as constants at the top of this file for easy tuning

# ---- PROMPT TEMPLATES (teammate fills these in) ----

SYSTEM_PROMPT = """
# STUB – teammate implements this
# Should instruct the model to:
# 1. Act as a senior security analyst
# 2. First write a PROSECUTION section: build the risk case with specific evidence
# 3. Then write a DEVIL'S ADVOCATE section: challenge the prosecution, consider alternative explanations
# 4. Output a structured JSON with prosecution, devils_advocate, doubt_score, final_severity, and recommendation
# Context factors that increase doubt_score (reduce suspicion):
#   - job_title contains executive/CTO/VP/Director
#   - is_new_hire is True (within 30 days)
#   - department is Finance and action is export_data and timestamp is near month end
#   - privilege_level is service-account (no activity baseline exists)
#   - days_inactive < 7 (recently active)
# doubt_score thresholds:
#   0.0 - 0.3 → final_severity = CRITICAL or HIGH
#   0.3 - 0.6 → final_severity = REVIEW
#   0.6 - 0.8 → final_severity = INFORMATIONAL
#   0.8 - 1.0 → final_severity = SUPPRESSED
"""

USER_PROMPT_TEMPLATE = """
# STUB – teammate implements this
# Should inject the user profile and flagged events as structured context
# Include: username, department, job_title, privilege_level, days_inactive, is_new_hire
# Include: risk_score, flags, sod_violations, compliance_gaps
# Include: flagged_events list (timestamp, action, resource, resource_sensitivity)
# Include: feedback examples from feedback.json as few-shot context if available
"""

def generate_narrative(user_profile: dict, flagged_events: list[dict], feedback_examples: list[dict]) -> dict:
    # Takes a UserRiskProfile, the user's flagged events, and feedback context
    # Makes a single Anthropic API call with the combined prosecutor + DA prompt
    # Parses the JSON response
    # Returns narrative dict:
    # {
    #   prosecution: str,
    #   devils_advocate: str,
    #   doubt_score: float,
    #   final_severity: str,      # "CRITICAL" | "HIGH" | "REVIEW" | "INFORMATIONAL"
    #   recommendation: str,
    # }

```

```
# breach_impact: str,
# }
# STUB: return hardcoded narrative dict
return {
    "prosecution": f"{user_profile['username']} has accessed high-sensitivi
    "devils_advocate": f"User is in {user_profile['department']} department
    "doubt_score": 0.4,
    "final_severity": "REVIEW",
    "recommendation": "Verify access patterns with direct manager before es
    "breach_impact": f"If compromised, attacker could access: {'', '}.join(us
}

def stream_narrative(user_profile: dict, flagged_events: list[dict], feedback_c
# Generator function – yields narrative text token by token for StreamingRe
# STUB: yields the full stub narrative in one chunk
# REAL: use anthropic client.messages.stream() and yield each text delta
narrative = generate_narrative(user_profile, flagged_events, feedback_conte
full_text = f"PROSECUTION:\n{narrative['prosecution']}\n\nDEVIL'S ADVOCATE:
yield f"data: {full_text}\n\n"
yield "data: [DONE]\n\n"
```

backend/pipeline.py

python

```
# Orchestrator – runs the full ML pipeline in order
# This is the only file that imports from all models/
# Order of operations is critical – do not reorder

import threading

# Global pipeline state – used by /status endpoint
pipeline_state = {
    "status": "idle",          # "idle" | "running" | "complete" | "error"
    "progress": 0,             # 0 to 100
    "message": "",             # human readable current step
    "last_run": None,          # ISO timestamp of last completed run
}

def run_pipeline() -> None:
    # Full pipeline – called in a background thread from /run-pipeline endpoint
    # Steps in order:
    # 1. Set pipeline_state status=running, progress=0, message="Loading data"
    # 2. data_loader.load_users() -> raw_users
    # 3. data_loader.load_events() -> raw_events
    # 4. Convert each raw_user to UserRiskProfile using schemas.make_stub_profi
    # 5. Set progress=20, message="Scoring users"
    # 6. isolation_forest.score_users(profiles, raw_events) -> updates profiles
    # 7. Set progress=40, message="Clustering events"
    # 8. kmeans_events.cluster_events(raw_events) -> event_clusters
    # 9. kmeans_events.assign_cluster_to_users(profiles, event_clusters) -> upda
    # 10. Set progress=60, message="Detecting org anomalies"
    # 11. org_anomaly.detect_org_anomalies(profiles) -> updates profiles
    # 12. Set progress=75, message="Checking SoD violations"
    # 13. For each profile: sod_violations.check_sod_violations(profile, user_e
    # 14. Set progress=88, message="Computing compliance gaps"
    # 15. For each profile: compliance.compute_compliance_gaps(profile)
    # 16. Set progress=95, message="Saving results"
    # 17. data_loader.save_profiles(profiles)
    # 18. Set progress=100, status=complete, message="Pipeline complete", last_
    pass

def run_pipeline_async() -> None:
    # Starts run_pipeline() in a background thread
    # Sets pipeline_state status=running immediately before starting thread
    # Use threading.Thread(target=run_pipeline, daemon=True).start()
    pass
```

Define these FastAPI endpoints with comments on each. Use proper HTTP status codes. Include CORS middleware allowing all origins (for local dev):

```
python

# POST /run-pipeline
# Triggers pipeline_async(), returns {"status": "started"} immediately
# If pipeline already running, return 409 Conflict

# GET /status
# Returns pipeline_state dict
# Used by frontend for polling

# GET /users/risks
# Loads profiles from data_loader.load_profiles()
# Applies feedback from data_loader.load_feedback() – sets suppressed field on
# Sorts by risk_score descending
# Returns list of UserRiskProfile dicts

# GET /users/{user_id}
# Returns single UserRiskProfile by user_id
# 404 if not found

# GET /users/{user_id}/narrative (SSE endpoint)
# Checks if profile already has narrative cached (not None)
# If cached: streams it back as SSE
# If not cached: calls llm.narrator.stream_narrative(), streams tokens back, ca
# Uses FastAPI StreamingResponse with media_type="text/event-stream"
# After streaming complete, saves updated profile back to user_profiles.json

# POST /feedback/{user_id}
# Body: { "action": "suppress" | "unflag", "note": str }
# Loads feedback.json, updates entry for user_id, saves back
# Also updates suppressed field on the profile in user_profiles.json
# Returns {"status": "ok"}

# GET /org-anomalies
# Loads profiles, filters to profiles where org_anomaly.is_anomaly == True
# Returns list of { user_id, username, department, risk_score, org_anomaly } di

# GET /stats
# Returns summary stats for dashboard header:
# { total_users, critical_count, high_count, review_count, suppressed_count,
#   avg_risk_score, top_risky_departments: list of {dept, avg_score} }
```

frontend/src/api.js

Define all fetch functions. Every function must have a comment explaining what it calls and what it returns. Use `async/await`. Base URL is `http://localhost:8000`.

javascript

```
// runPipeline() → POST /run-pipeline → returns {status}
// getStatus() → GET /status → returns pipeline_state dict
// getUserRisks() → GET /users/risks → returns array of UserRiskProfile
// getUser(userId) → GET /users/{userId} → returns single UserRiskProfile
// getFeedback is handled server-side, no direct call needed
// postFeedback(userId, action, note) → POST /feedback/{userId} → returns {stat
// getOrgAnomalies() → GET /org-anomalies → returns array
// getStats() → GET /stats → returns stats dict
// streamNarrative(userId, onChunk, onDone) → opens EventSource to GET /users/{
//   calls onChunk(text) for each SSE chunk, calls onDone() when [DONE] receive
```

frontend/src/App.jsx

jsx

```
// Root component – single page app
// State:
//   pipelineStatus: string ("idle" | "running" | "complete" | "error")
//   users: array of UserRiskProfile (loaded from /users/risks after pipeline c
//   stats: object (from /stats)
//   selectedUser: UserRiskProfile | null (set when a row is clicked)
//   activeTab: string ("risks" | "org" | "compliance")
//
// On mount:
//   - fetch /status to check if pipeline already ran (status === "complete")
//   - if complete: load users and stats immediately
//   - start polling /status every 5 seconds
//   - when status transitions to "complete": load users and stats
//
// Layout (Tailwind):
//   - Top: StatusBar component (shows pipeline status + Run Pipeline button)
//   - Stats header row: 5 stat cards (total users, critical, high, review, avg
//   - Tab bar: "Risk Table" | "Org Anomalies" | "Compliance"
//   - Main content area: renders active tab component
//   - Right panel (slide-in): UserCard when selectedUser is not null
```

frontend/src/components/StatusBar.jsx

```
jsx

// Props: { status: string, progress: number, message: string, onRunPipeline: f
// Shows current pipeline status with a progress bar when running
// "Run Pipeline" button – disabled when status is "running"
// Shows last run timestamp when status is "complete"
// Color coding: idle=gray, running=blue, complete=green, error=red
```

frontend/src/components/RiskTable.jsx

```
jsx

// Props: { users: array, onSelectUser: fn }
// Table with columns: Username | Department | Privilege | Risk Score | Flags |
// Risk Score shown as colored badge: >=80 red, >=60 orange, >=40 yellow, <40 g
// Flags shown as small pill badges (STALE_ADMIN, SOD_VIOLATION, etc.)
// Suppressed users shown with strikethrough and gray styling
// Clicking a row calls onSelectUser(user)
// Sortable by risk_score (default desc)
// Filterable by department and privilege_level (dropdown filters above table)
// Comment: // Suppressed users are still shown but visually de-emphasized – an
```

frontend/src/components/UserCard.jsx

```
jsx
```

```
// Props: { user: UserRiskProfile, onClose: fn }
// Slide-in panel showing full user detail
// Sections:
// 1. Header: username, email, department, privilege level badge, risk score
// 2. Identity details: systems_access chips, hire_date, days_inactive, is_ne
// 3. Event summary: event_count, after_hours_ratio bar, high_sensitivity_rat
// 4. Flags section: list of flag codes with descriptions
// 5. NarrativePanel component (lazy – only renders when user is selected)
// 6. BreachImpactPanel component
// 7. CompliancePanel component
// 8. FeedbackButtons component
// Comment: // NarrativePanel triggers SSE stream on mount – do not mount until
```

frontend/src/components/NarrativePanel.jsx

```
jsx

// Props: { userId: string, cachedNarrative: object | null }
// On mount: if cachedNarrative exists, display it immediately
// If not cached: open SSE stream via api.streamNarrative()
// Show streaming text progressively as tokens arrive
// Structure:
// - "PROSECUTION" section (red/orange tinted card)
// - "DEVIL'S ADVOCATE" section (blue tinted card) – appears after prosecutio
// - Doubt score meter: horizontal bar 0.0 to 1.0 with color gradient red→gre
// - Final severity badge: CRITICAL=red, HIGH=orange, REVIEW=yellow, INFORMAT
// - Recommendation box (green tinted)
// Loading state: show animated pulse skeleton while streaming
// Comment: // doubt_score above 0.6 means DA won – severity is downgraded, sho
```

frontend/src/components/OrgAnomalyPanel.jsx

```
jsx

// Props: { anomalies: array } – array from GET /org-anomalies
// Groups anomalies by department
// Shows a bar chart (Recharts BarChart) of avg risk score per department
// Below chart: list of flagged users with their org_anomaly.reason
// Comment: // This panel is shown in the "Org Anomalies" tab, not inside UserC
```

frontend/src/components/BreachImpactPanel.jsx

```
jsx

// Props: { user: UserRiskProfile }
// Shows breach_impact string from narrative (if available)
// Also shows systems_access as an impact map:
//   List of systems the user can access, each with a sensitivity indicator
//   (infer sensitivity from system name: PROD_DB=critical, HRIS=high, BI_Tool=
// Comment: // breach_impact is populated by LLM – show loading state if narrat
```

frontend/src/components/CompliancePanel.jsx

```
jsx

// Props: { user: UserRiskProfile }
// Shows user.compliance_gaps as a list of cards
// Each card: framework badge (NIST/GDPR), control_id, control_name, gap descri
// If compliance_gaps is empty: show "No compliance violations detected" in gre
// Comment: // compliance_gaps computed by backend rules engine – no LLM call n
```

frontend/src/components/FeedbackButtons.jsx

```
jsx

// Props: { user: UserRiskProfile, onFeedbackSubmit: fn }
// Two buttons: "Mark as False Positive" (suppress) and "Confirm Threat" (unfla
// When suppress clicked: show text input for analyst note, then POST /feedback
// Visual state: if user.suppressed === true, show "Suppressed" badge and "Re-f
// onFeedbackSubmit called after successful API call so parent can refresh user
// Comment: // Suppressing a user adds them to feedback.json which is injected
```

ADDITIONAL INSTRUCTIONS FOR GEMINI

1. **Generate every file completely.** Do not write partial files. Do not write "// rest of implementation here". Every function must have either real logic or a complete stub that returns the correct shape.
2. **Add comments generously.** Every function needs a docstring or comment block. Every non-obvious line needs an inline comment. Every stub must have a comment

that says `# STUB – replace with real implementation` and ideally a comment describing what the real implementation should do.

3. **The stubs must return correct shapes.** A stub that returns `None` or `{}` or `[]` is not acceptable if the real function returns a specific shape. The stub must return a realistic example of the correct shape so the frontend renders something meaningful.
4. **Do not use any library not listed in the tech stack.** No SQLAlchemy, no Redis, no Celery, no additional React libraries beyond what is listed.
5. **Backend must start with:** `uvicorn main:app --reload --port 8000`
6. **Frontend must start with:** `npm run dev` (Vite default port 5173)
7. **Include a** `requirements.txt` **for the backend with pinned major versions:** fastapi, uvicorn, pandas, scikit-learn, numpy, anthropic
8. **Include** `package.json` **with all frontend deps listed:** react, react-dom, recharts, lucide-react, tailwindcss, autoprefixer, postcss, @vitejs/plugin-react
9. **CORS:** Backend must allow `http://localhost:5173` explicitly in FastAPI CORS middleware.
10. **The pipeline must auto-run on backend startup** if `user_profiles.json` does not exist. If it exists, load from file and skip recompute. Add a comment explaining this.
11. **Tailwind dark theme.** Use `dark` class on the root div. Background `bg-gray-950`, cards `bg-gray-900`, text `text-gray-100`. Risk score colors: critical=`text-red-400`, high=`text-orange-400`, review=`text-yellow-400`, info=`text-blue-400`.
12. **Do not create the CSV files.** They already exist at `backend/data/identity_users.csv` and `backend/data/identity_events.csv`.
13. **The feedback loop must be real even in the skeleton.** The `POST /feedback/{user_id}` endpoint must actually write to `feedback.json` and the `GET /users/risks` endpoint must actually apply suppressed status from that file. This is not stubbed — it is real from day one.
14. **File separator comments.** At the top of every file, add a comment block:

```
# =====  
# FILE: backend/models/isolation_forest.py  
# OWNER: [Person 1 – ML pipeline]  
# STATUS: STUB – replace internals with real implementation  
# DEPENDS ON: schemas.py, data_loader.py  
# =====
```

For frontend files use `//` comments. The OWNER field should say "Person 1 — ML pipeline" for `isolation_forest.py`, `kmeans_events.py`, `org_anomaly.py`, `sod_violations.py`, `compliance.py`, `pipeline.py`, `data_loader.py`. "Person 2 — LLM + UI" for `narrator.py` and all frontend files.

WHAT TO GENERATE

Generate every file listed in the folder structure above, in this order:

1. `backend/requirements.txt`
2. `backend/schemas.py`
3. `backend/data_loader.py`
4. `backend/models/__init__.py` (empty)
5. `backend/models/isolation_forest.py`
6. `backend/models/kmeans_events.py`
7. `backend/models/org_anomaly.py`
8. `backend/models/sod_violations.py`
9. `backend/models/compliance.py`
10. `backend/llm/__init__.py` (empty)
11. `backend/llm/narrator.py`
12. `backend/pipeline.py`
13. `backend/main.py`
14. `frontend/package.json`
15. `frontend/vite.config.js`
16. `frontend/tailwind.config.js`
17. `frontend/postcss.config.js`
18. `frontend/index.html`
19. `frontend/src/main.jsx`
20. `frontend/src/api.js`
21. `frontend/src/App.jsx`
22. `frontend/src/components/StatusBar.jsx`
23. `frontend/src/components/RiskTable.jsx`
24. `frontend/src/components/UserCard.jsx`
25. `frontend/src/components/NarrativePanel.jsx`
26. `frontend/src/components/OrgAnomalyPanel.jsx`
27. `frontend/src/components/BreachImpactPanel.jsx`
28. `frontend/src/components/CompliancePanel.jsx`
29. `frontend/src/components/FeedbackButtons.jsx`